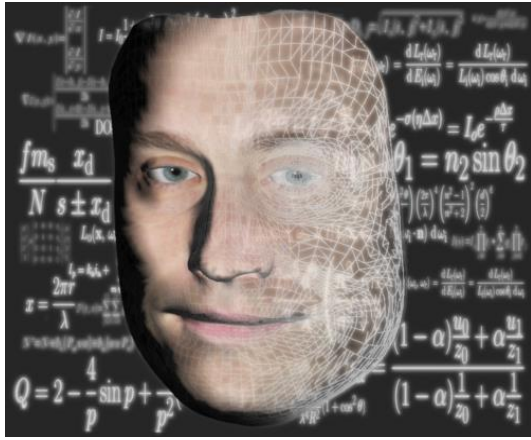




Object Detection & Counting on Raspberry Pi:

Classical OpenCV and Lightweight ResNet Comparison

Emmad Ismail, Nico Pschipsch, Douglas W. Cunningham,
Department of Graphical Systems, BTU Cottbus-Senftenberg, Cottbus, Germany

1. INTRODUCTION

How many objects are in an image?
Can a low-cost computer answer that reliably?

Object detection and counting has applications from traffic monitoring to warehouse automation. *Abdulhamid et al. [1]* built a low-cost Raspberry Pi system achieving **90.206%** counting accuracy on simple shapes using OpenCV. However, this classical approach struggles when objects are close together, poorly lit, or affected by shadows, and cannot distinguish circular shapes at all.

2. RESEARCH FOCUS

Following *Abdulhamid et al. [1]*, this project asks: does a machine learning approach surpass classical image processing for object counting?

- The classical pipeline will replicate 90.206% accuracy under clean conditions, but degrade under challenging ones.
- A ResNet model will outperform the classical pipeline specifically under challenging conditions, since learned features generalize better than hand-crafted edges.

4. RESULTS

Table 1. Key Results:

Metric	Value	Outcome
ResNet — Kaggle validation accuracy	98.6%	Transfer learning correctly learned to separate the 3 shape classes
Track A — real-world counting efficiency	61.28%	Real, cluttered scenes are far harder than the paper's clean, cropped images (paper reported 90.206%)
Track A vs Track B — real-world classification accuracy	48.1% / 59.1%	ResNet ahead overall, but the gap is not uniform (see Fig. 3)

Table 2. Key Findings — breakdown by object type:

Object Type	Track A	Track B	Gap
3D solid objects (pyramids, cubes, spheres, cylinders)	34.5%	66.4%	+31.9 pts
Flat 2D cutouts	56.0%	54.9%	-1.1 pts

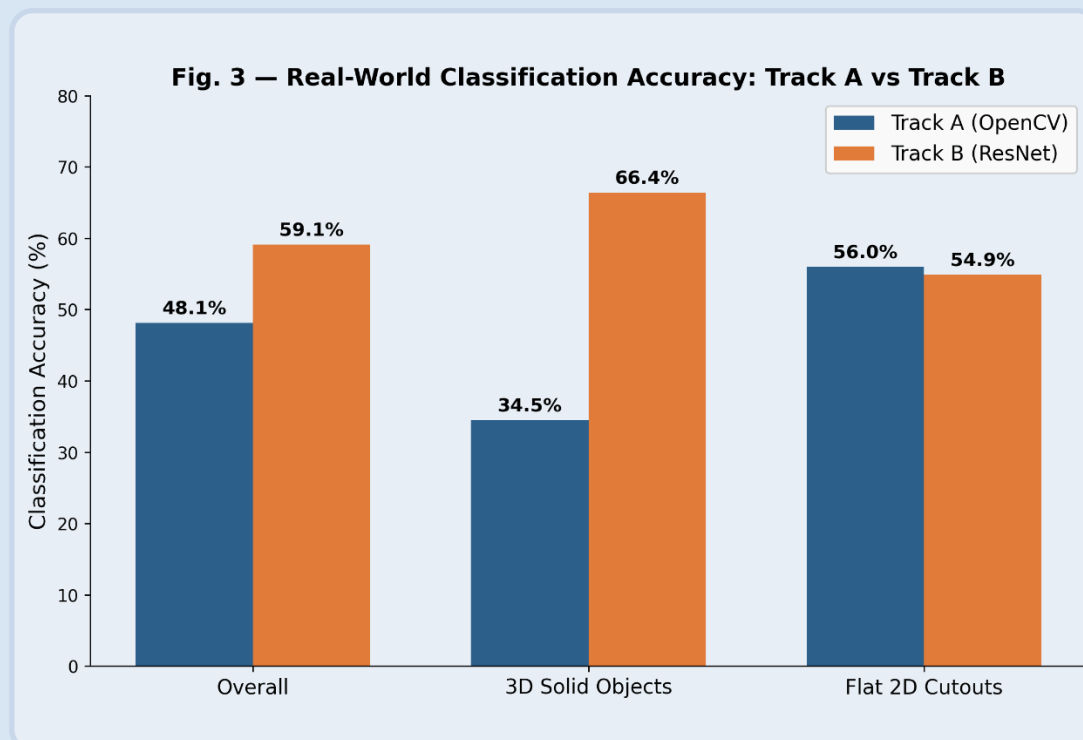


Fig. 5. Real-world classification accuracy, Track A vs Track B, overall and split by object type (bar chart).

5. EVALUATION

- Ground truth:** True object count and shape recorded manually per photo by the experimenter.
- Counting metric:** Efficiency = $1 - |\text{detected} - \text{true}| / \text{true}$, per photo, same formula as *Abdulhamid et al. [1]*, enabling direct comparison to their 90.206% baseline.
- Classification metric:** Since Track A and B share detection, accuracy is scored via label-overlap against true shapes.
- Test set:** 38 real photos — 14 clean / 24 close-spacing; 14 with 3D solid objects / 24 with flat 2D cutouts.

6. LIMITATIONS

- Vertex-counting vs. 3D objects at an angle:** a cube viewed obliquely shows 5–6+ outline vertices, not 4. Track A cannot succeed here regardless of image quality. Structural, not tunable (confirmed: 34.5% vs 56.0% for Track A, 3D vs flat objects).
- Horizon-line ambiguity:** where an object's base touches the table/backdrop boundary, no pixel-level rule can reliably separate touching objects from one merged shape.

References:

- [1] Abdulhamid, M., Odoni, O., & Al-Rawi, M. (2020). Computer Vision Based on Raspberry Pi System. *Applied Computer Science*, 16(4), 85–102.
[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
[3] Piyasara, D. Geometric Shapes Dataset. Kaggle. <https://www.kaggle.com/datasets/dineshpiyasamara/geometric-shapes-dataset>

3. METHODOLOGY

This project follows a two-track approach. Track A reimplements the classical OpenCV pipeline from *Abdulhamid et al. [1]* as the baseline, extended with additional preprocessing steps to handle real, uncropped room photos (table segmentation, background exclusion, lighting-adaptive thresholding). Track B reuses Track A's detection step and replaces only the shape-classification stage with a ResNet CNN, isolating the classification-only comparison.

Dataset:

- Training data (Track B):** Kaggle *geometric-shapes-dataset* [3] — 10,000 images per class (Circle, Square, Triangle). Pretrained ResNet18 fine-tuned via transfer learning (1,500 training / 300 validation images per class; 98.6% validation accuracy).
- Test data (both tracks):** 38 self-captured photos, Raspberry Pi camera, physical 2D and 3D objects (cubes, pyramids, spheres, cylinders, flat cutouts), fixed raspberry pi camera setup.
- Conditions present:** clean (well-spaced), close (tightly spaced/touching objects).

Note on terminology: Kaggle's "Square" class includes only perfect squares; random aspect-ratio augmentation helped generalize to real non-square rectangles.

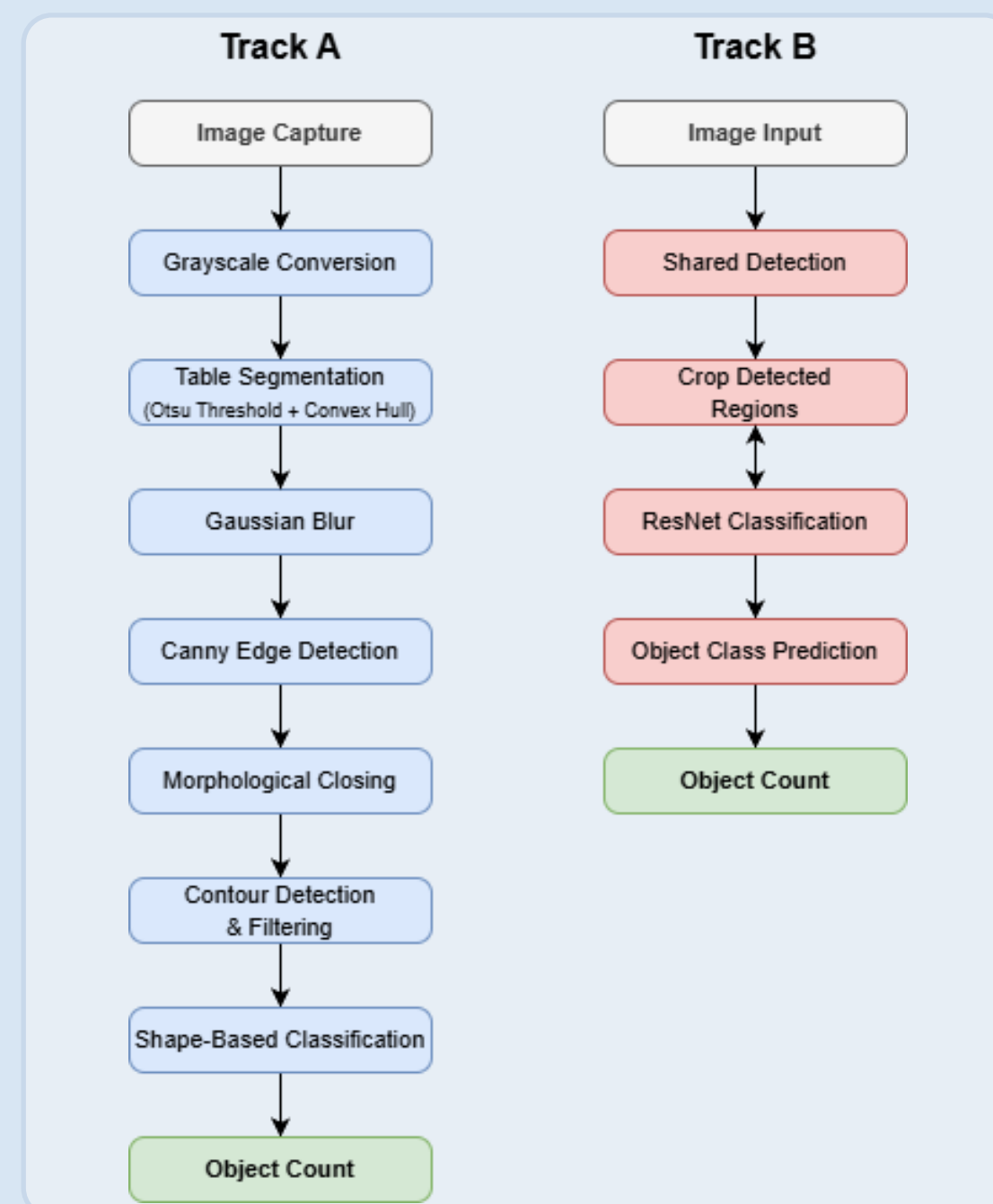


Fig. 1. Two-track methodology: Track A (classical OpenCV baseline, adapted from Abdulhamid et al. [1]) and Track B (ResNet-based extension, inspired by He et al. [2]), sharing a common detection stage.

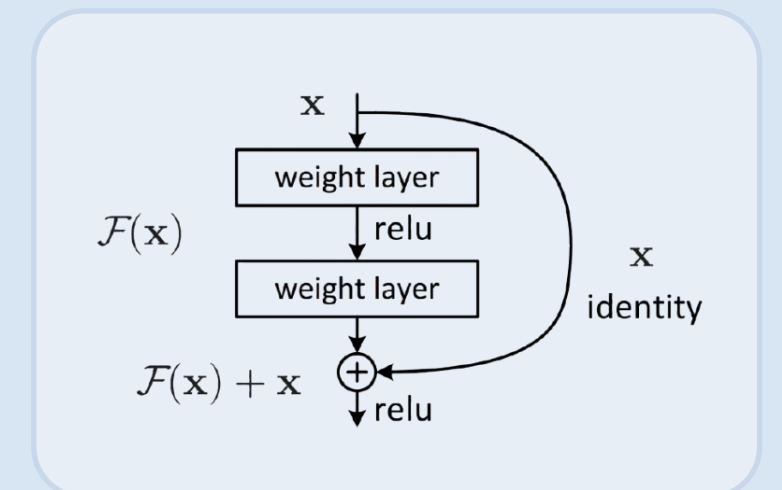


Fig. 2. Residual learning block with an identity in the ResNet architecture, adapted from He et

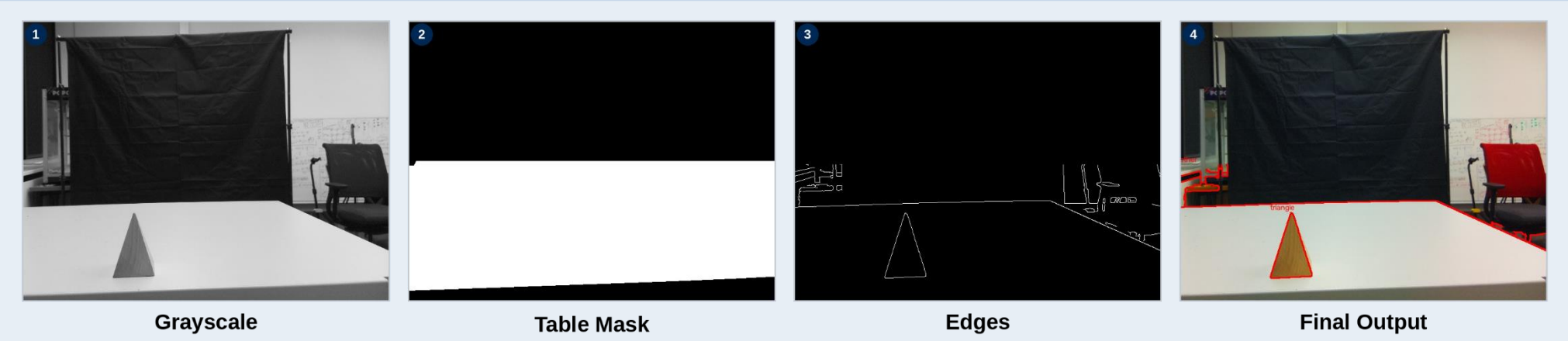


Fig. 3. Track A's OpenCV pipeline applied to a real test photo, shown stage by stage: grayscale conversion, table segmentation (Otsu threshold + convex hull), Canny edge detection, and the final shape-classified output with bounding contour and label.

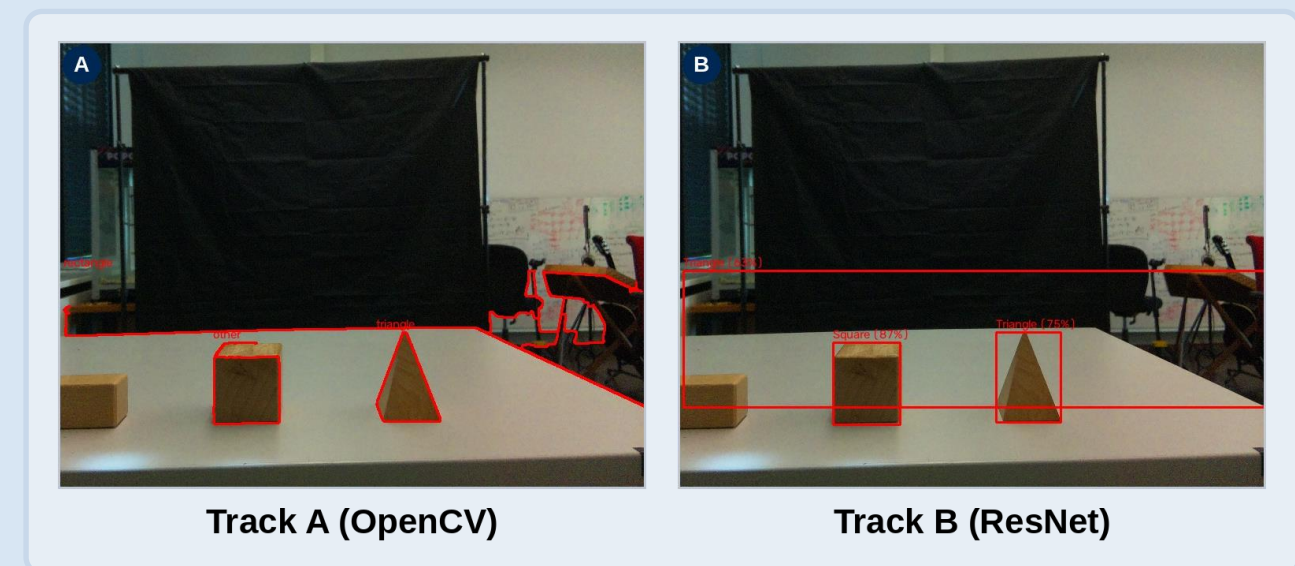


Fig. 4. Track A (OpenCV, left) vs Track B (ResNet, right) on the same photo. Track A misclassifies the cube as "other," while Track B correctly classifies it as "Square" → rectangle (87% confidence) — the same 3D-object gap reflected in Table 2. Both tracks share Track A's detection stage, so the background contour and undetected front block appear identically in both panels.

7. CONCLUSION/FUTURE WORK

Both hypotheses were confirmed. H1 as predicted: the classical pipeline degraded from ~90% (clean, controlled) to 61.28% (real-world). H2 more precisely than predicted: ResNet's advantage isn't general to "challenging conditions". It's concentrated specifically on 3D objects viewed obliquely (34.5% vs 66.4%), while flat 2D cutouts remain statistically tied (56.0% vs 54.9%).

Future work: give Track B independent detection (not shared with Track A); test genuine shadow conditions; generalize beyond a fixed camera position.